Unit 5 Assessment

See Part 1 at https://github.com/tombracey/Unit-5-Assessment

2. a. 'An explanation of the products, tools and methodologies used to facilitate CI/CD'

Overview of CI/CD

CI/CD is a methodology and set of processes which are ubiquitous in modern software development. Together they can make the development of an application more seamless and secure. The aim is also to speed up the development.

The CI in CI/CD stands for Continuous Integration. This is about ensuring code changes are always validated automatically before they are merged into a shared Git branch. Depending on how the CI is set up, the CI process could be triggered by each commit, each push, or when the Git branch is merged into the main (or master) branch.

The CD in CI/CD stands for Continuous Deployment, but is always sometimes referred to as 'Continuous Delivery'. This is about ensuring the tested code is ready to be deployed to the production environment.

Tools for CI/CD

The first CI/CD tool with mass adoption was Jenkins, an open-source automation server. Typically, this would be hosted locally be internal infrastructure. The open-source nature meant that it's highly customisable as the Jenkins community-built plugins to cater for different automation needs.

GitHub Actions is probably the most widely used CI/CD tool today as it's integrated within GitHub and offers a generous free allowance to run automated CI workflows on their servers. YAML workflow files are used to define jobs, and CRON syntax is used for scheduling.

The main CI/CD tool used in my company is GoCD, another open-source tool with extensibility. The benefit of GoCD within a microservice architecture is the clear overview it provides for all the company's apps.

CI/CD Focuses in My Team

The most important focus of CI/CD in my team is testing. Testing is a huge focus in my company's engineering culture more broadly, and our CI/CD processes allow us to run tests automatically which ensures new code does what it intends and doesn't break any of the logic elsewhere. If a test fails in one of our GoCD pipelines, this issue is identified before the new code can reach the production environment.

It's one thing to make sure all the existing unit tests are passing, but it's another to make sure everything that should be tested is being tested. For this we use test coverage tools as part of our CI/CD process. In a recent Java project, I used the 'JaCoCo Java Code Coverage Library', which checks for branch coverage - that is, if there are logical branches within the codebase like within an if statement or switch statement, then there must be a unit test for each possible path.

Another concern of our CI/CD processes is linting. In shared repositories it's important that one person's code is legible and maintainable to all the other people on the team, and the best way to ensure everyone is linting their code to the same standards and requirements is to make it automatic. For backend Java projects we use Checkstyle, and for frontend Typescript projects we use ESLint.

Finally, a vital function of CI/CD within our live applications is automatically identifying vulnerabilities within new or existing code. Tools such as the 'safety' package in Python can scan a project's imports for any Common Vulnerabilities and Exposures (CVEs).

b. 'An evaluation of what you personally find useful, and why, with regards to 2a'

Personal Evaluation of CI/CD Within My Team

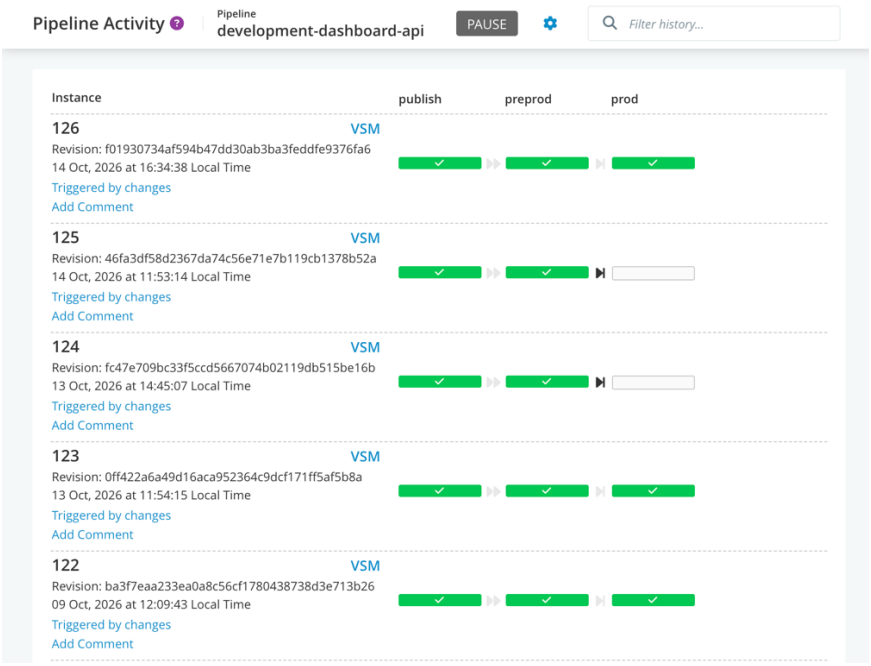
Evaluating these tools and methodologies, I personally find the automated safety net provided by GoCD to be the most valuable aspect of our CI/CD process. As a team of apprentices, we are continuously learning, and the risk of introducing breaking changes is a natural part of that learning curve.

c. 'Evidence of your use of CI/CD in the form of annotated screenshots'

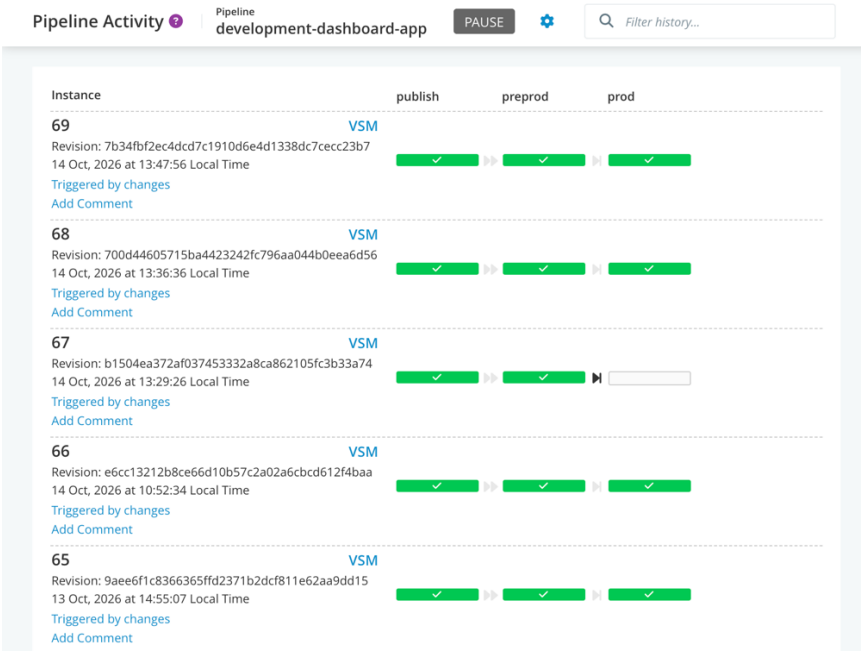
Evidence of Use of CI/CD

I recently built an application called the 'Development Dashboard' with my team. The app was split into two repositories – one for the frontend and one for the backend – which both used GoCD pipelines for CI/CD.

Backend pipeline ('Development Dashboard API')



Frontend pipeline (‘Development Dashboard App’)



The frontend was established as a separate application with its own GoCD pipeline because it has a Backend for Frontend (BFF) in the form of a lightweight Node server making requests to the backend API, which is a concept that is being rolled out across most frontend repos in the company. Adding CI/CD to frontend repositories means that validation of code changes and deployment have the same robust quality controls as the backend repositories.

d. A description of how (/if?) testing is achieved through CI/CD principles and tools

How Testing is Achieved Through CI/CD Principles and Tools

Testing is achieved through CI/CD within my team as test coverage is enforced within the process (usually 80% or 90% required to push changes). Additionally, all unit tests must pass in order for new changes to be merged to master and run in a pre-prod and prod environments.